

M6. 파이썬으로 다시 보기

스프레드시트로 하던 일을 코드로

1부 정리 — 스프레드시트로 한 일

1. M1. 시간이 담긴 데이터 — 시계열, 꺾은선 그래프
2. M2. 평균의 힘과 함정 — 평균과 중앙값
3. M3. 추세 찾기 — 이동평균
4. M4. 반복되는 패턴 — 계절성
5. M5. 첫 예측 — naive 예측과 이동평균 예측

이 다섯 모듈을 전부 스프레드시트로 했다. 오늘부터는 같은 일을 코드로 한다.

오늘의 목표

- Colab에서 노트북 셀을 실행할 수 있다.
- pandas로 CSV 파일을 불러올 수 있다.
- M1-M3에서 한 일(그래프.평균.이동평균)을 코드로 재현할 수 있다.

왜 파이썬인가 — 1,000일도 한 줄

- 시트에서 누른 클릭들은 기록이 남지 않는다. 코드는 요리 레시피처럼 남는다 — 누가 다시 실행해도 같은 결과가 나온다.
- 우리가 쓰는 데이터는 1,096일치다. 이 정도라 시트가 버텼지만, 데이터가 100만 줄이면? 코드는 여전히 한 줄이다.
- 오늘 1부 전체를 코드 몇 줄로 다시 해본다.

Colab이란

- 설치 없이 브라우저에서 쓰는 파이썬 공책(노트북).
- 구글 계정만 있으면 바로 쓸 수 있다.
- colab.research.google.com에 접속해 새 노트북을 만든다.

셀 실행 방법 — Shift+Enter

- 노트북의 빈 칸이 셀이다.
- 셀에 코드를 쓰고 **Shift+Enter**를 누르면 실행되고, 바로 아래에 결과가 나온다.
- 노트북은 위에서부터 순서대로 실행하는 것이 규칙이다.

첫 셀 — 파이썬과 인사

```
print("안녕, 시계열!")
```

파이썬은 계산기이기도 하다.

```
(1 + 2 + 2 + 3 + 42) / 5
```

이 숫자 다섯 개, 어디서 본 조합일까? (힌트: M2의 첫 활동) — 직접 실행해서 확인해 보자.

pandas란

- 표(데이터프레임)를 다루는 파이썬 도구 상자.
- `import pandas as pd` 로 불러와 `pd` 라는 이름으로 쓴다.
- 시트에서 표를 다루던 것을, pandas에서는 코드로 다룬다.

CSV 읽기 — 데이터 불러오기

```
import pandas as pd

url = "https://raw.githubusercontent.com/jaehoonkim/timeseries-academy/main/data/seoul-temp-daily.csv"
df = pd.read_csv(url, parse_dates=["date"])
df
```

따옴표.괄호까지 정확히 입력해야 한다.

한 줄씩 읽는 법

코드	하는 일	시트에서는
<code>import pandas as pd</code>	표 전용 도구 상자를 꺼내 <code>pd</code> 라고 부르기	(시트를 여는 것)
<code>pd.read_csv(url, ...)</code>	주소에서 CSV 읽어 오기	<code>=IMPORTDATA(...)</code> — 같은 주소 다!
<code>parse_dates=</code> <code>["date"]</code>	date 열을 날짜로 인식시키기	서식 → 숫자 → 날짜 (M1의 45139 사건)
<code>df = ...</code>	읽어 온 표를 <code>df</code> 라는 이름에 담기	(시트 문서 그 자체)

확인해 보기 — len과 tail

```
len(df)
```

```
df.tail()
```

`len(df)` 는 표의 행 수, 즉 데이터가 며칠치인지 알려준다. `tail()` 은 표의 끝부분 몇 줄을 보여준다 — 결과는 활동지에서 직접 확인한다.

M1 재현 — 그래프 한 줄

```
df.plot(x="date", y="temp_avg")
```

범위 선택 → 삽입 → 차트로 몇 번 클릭하던 일이 한 줄이 된다.

```
df.plot(x="date", y=["temp_avg", "temp_max", "temp_min"])
```

y 에 열 이름을 리스트로 주면 선을 여러 개 한 번에 그린다.

M2 재현 — 평균과 중앙값

`df["열이름"]` 에 함수를 붙인다. 새 셀마다 하나씩 실행한다.

```
df["temp_avg"].mean()
```

```
df["temp_avg"].median()
```

```
df["temp_avg"].max()
```

```
df["temp_avg"].min()
```

max·min으로 상식적인 범위인지 확인하는 것도 코드로 할 수 있다.

M3 재현 — 이동평균

```
df["temp_avg"].rolling(window=7).mean()
```

= temp_avg에 / 7일짜리 창을 하루씩 밀며(rolling) / 창마다 평균

```
df["ma7"] = df["temp_avg"].rolling(window=7).mean()  
df.head(10)
```

`rolling(window=7)` 은 M3에서 손으로 밀던 바로 그 창이다. 표 앞부분에는 계산할 데이터가 모자라 **NaN**(값 없음)이 나온다 — 무엇과 같은 자리일지 활동지에서 확인해 보자.

에러를 만나면 — 에러는 정상

- **NameError: name 'df' is not defined** — 위 셀을 실행하지 않고 아래 셀부터 실행한 것. 노트북은 위에서부터 순서대로.
- **SyntaxError** — 따옴표, 괄호 짝이 안 맞는 것. 에러 메시지의 화살표(^)가 위치를 가리켜 준다.
- 대소문자도 구분한다 — `df` 와 `Df` 는 다른 이름이다.
- 에러 메시지는 혼내는 게 아니라 힌트다.

오늘 배운 세 문장

1. 셀에 쓰고 Shift+Enter가 전부다.
2. 시트의 클릭 하나는 코드 한 줄과 짝이 있다.
3. 에러 메시지는 힌트다.

이제 활동지로

1. Colab에서 첫 셀을 실행해 본다.
2. CSV를 불러와 `len·tail`로 확인한다.
3. M1-M3을 코드로 재현한다 — 그래프, 평균, 이동평균.
4. 코드와 시트, 언제 뭐가 나enzi 생각해 본다.